

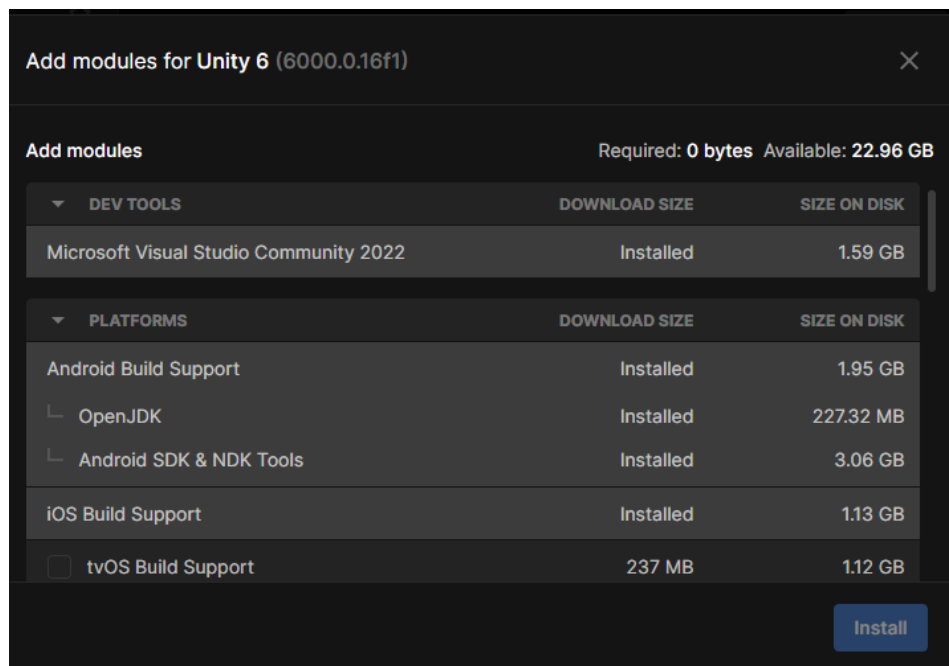
Guide d'installation

Installation	1
Unity Hub et Unity	1
Configuration d'un projet AR	1
Build l'application mobile	3
Mac et iOS	3
Windows et Android	3
Concepts clés Unity	4
Mots clés	4
Organisation des dossiers	5
Mots clés AR	5
Scène AR	5
Git et Unity	6
Boucle de rendu	6
Étapes	6
Logique	6
Liens Utiles	8

Installation

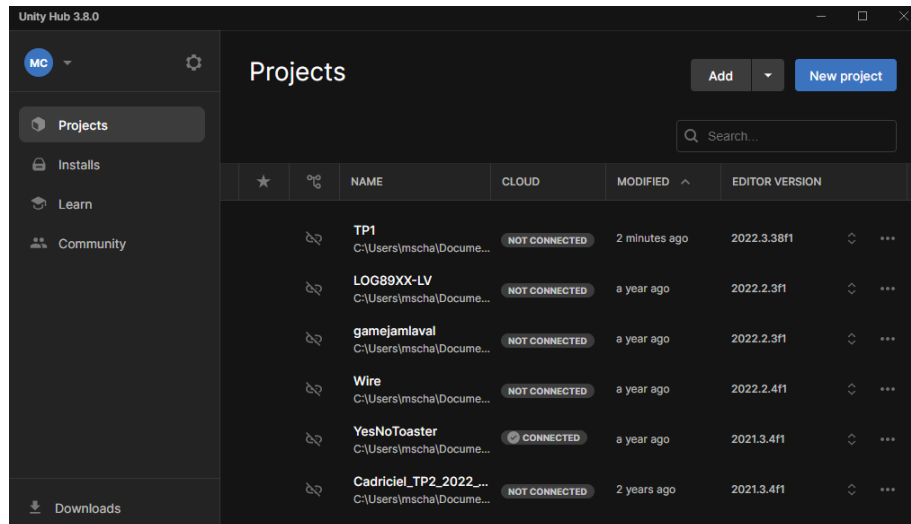
Unity Hub et Unity

1. Installer [Unity Hub](#).
2. Ouvrir Unity Hub et créer un compte Unity.
3. Installer Unity Editor
 - Dans l'onglet *installs*, cliquer sur *Install Editor* pour installer une version de Unity Editor. Installer la version Unity 6.
 - Les modules à sélectionner
 - Visual Studio
 - iOS Build Support (pour développer pour iOS)
 - Android Build Support (pour développer pour Android)

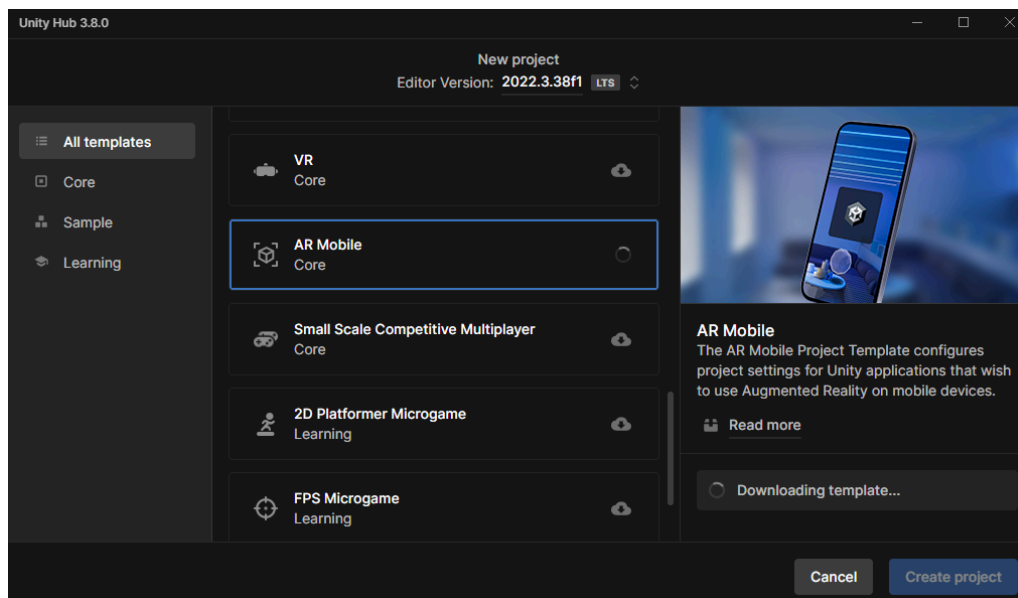


Configuration d'un projet AR

- Créer un nouveau projet
 - Aller dans l'onglet *Projects* et cliquer sur *New*.



- Télécharger et sélectionner le template *AR Mobile*. Ce template installe les packages nécessaires au développement AR:
 - AR Foundation
 - XR Interaction Toolkit
 - ARCore XR Plugin (pour Android)
 - ARKit XR Plugin (pour iOS)



- Nommer et créer le projet.
- Vérifier la scène AR

Build l'application mobile

Mac et iOS

1. [Build an iOS application](#)

Windows et Android

1. [Build an Android application](#)
2. Activer le mode développeur sur le téléphone
 - Aller dans les paramètres du téléphone et localiser le *Build Number*, appuyer plusieurs fois (7) jusqu'à ce qu'un message s'affiche pour confirmer que le mode développeur est activé.
 - Aller dans les options de développeur et s'assurer que le *USB debugging* est activé.
 - Connecter l'appareil à l'ordinateur.
3. *Build Settings* Unity
 - Aller à *File > Build Profiles* (ancien *Build Settings*)
 - Sélectionner Android et appuyer sur *Switch Platform*.
 - Sélectionner l'appareil mobile comme *Run Device*.
 - Sélectionner *Development Build*.
 - Appuyer sur *Build and Run*, créer un dossier Build et le sélectionner. Nommer le build.

Concepts clés Unity

Mots clés

- **Scène**
 - La scène est l'élément qui contient tout ce qui se passe dans l'application ou le jeu.
- ***GameObject***
 - Une classe de base qui définit tous les objets de la scène.
- ***Component***
 - Un élément qu'on ajoute à un *GameObject* pour lui ajouter une fonctionnalité.
- ***Scripts***
 - Permet de contrôler le comportement d'un objet. On l'attache à un *GameObject*, comme un *component*.
- ***Prefab***
 - Modèle d'un *GameObject* réutilisable. Lorsqu'on modifie un *Prefab*, toutes ses instances sont mises à jour. Pour créer un *Prefab*, glisser un *GameObject* de la scène dans un dossier du projet.
- ***Assets***
 - Ressource utilisée pour le projet. Un modèle 3D, une texture, un fichier audio, un script, etc.
- ***Physics***
 - Unity fournit des objets qui simulent des interactions physiques. Les composantes *Rigidbody* et *Collider* peuvent être ajoutés à un *GameObject* pour qu'ils agissent en suivant le système de physique de Unity.
- **Caméra**
 - Détermine la vue du joueur/l'utilisateur de l'application.
 - Il est possible d'avoir plusieurs caméras dans la scène, pour avoir différentes vues ou faire du *camera stacking*.
- **UI**
 - Système d'interface utilisateur avec lequel on peut écrire du texte, créer des boutons, des menus ou autres.
- ***Shaders et Materials***
 - Un *shader* définit le rendu des surfaces, leur interaction avec la lumière (s'il y a lieu), la texture et la couleur.
 - Un *Material* utilise un *shader* pour appliquer les propriétés de ce dernier à un *GameObject*. Il existe des *shaders* de base dans Unity.
 - Lorsqu'on modifie un *Material*, tous les *GameObject* qui y sont associés sont modifiés.
 - Pour créer un *Material*, faire un clic droit dans un dossier de l'onglet *project* et sélectionner *Create > Material*.
- [gitignore](#)

Organisation des dossiers

Une bonne pratique est d'organiser les *assets* d'un projet avec les dossiers suivants.

- Assets
 - Prefabs
 - Settings
 - Scene
 - Scripts
 - Materials
 - Textures

Mots clés AR

- AR Foundation
 - Bibliothèque qui facilite le développement d'applications AR en fournissant une interface commune pour plusieurs plateformes. Inclus le suivi de position, la détection de surfaces et la compréhension de l'environnement.
 - Pour déployer sur une plateforme spécifique, il faut le *provider plugin package*, ARCore ou ARKit par exemple.
- *Trackables*
 - Objets réels que l'application AR reconnaît et suit. Comme les surfaces planes, des visages, des images. etc.
- *Anchors*
 - Point fixe qui garde un objet virtuel en place même si l'utilisateur se déplace.
- *Plane Detection*
 - Fonctionnalité qui permet de détecter les surfaces planes (sol, tables)
- *Point Clouds*
 - Utilisés pour cartographier les surfaces ou pour le suivi.
- *Occlusion*
 - Capacité de cacher des objets virtuels derrière des objets réels. Aide à une intégration plus réaliste.

Scène AR

- **AR Session:** Gère le cycle de vie et les configurations des options AR de l'application.
- **XR Origin:** Gère la transformation des données XR en *world space*.
- **Object Spawner:** Gère les objets *interactables* de la scène.

Git et Unity

Git n'arrive pas toujours à bien *merge* les changements de scène. Il est donc recommandé de ne pas modifier les scènes en même temps ou de travailler dans des scènes différentes. Les *prefabs* sont aussi utiles pour palier à ce problème, modifier un *prefab* au lieu de l'objet dans la scène.

Boucle de rendu

La boucle de rendu représente le cycle de mises à jour et de rendu effectué par l'application.

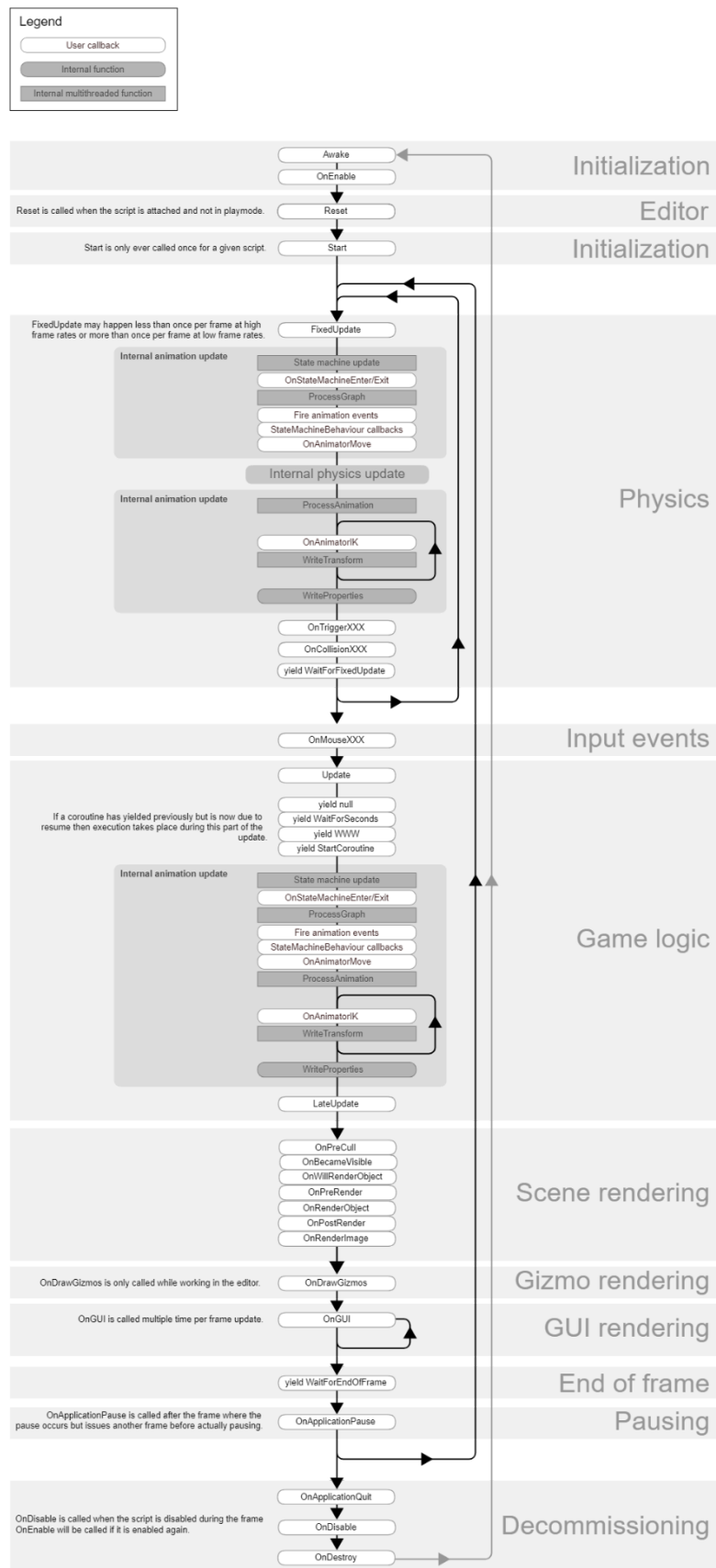
Étapes

1. Gestion des entrées
2. Update (mise à jour)
3. Résolution de la physique
4. *Rendering*
5. *Post-processing* (post-traitement)

Logique

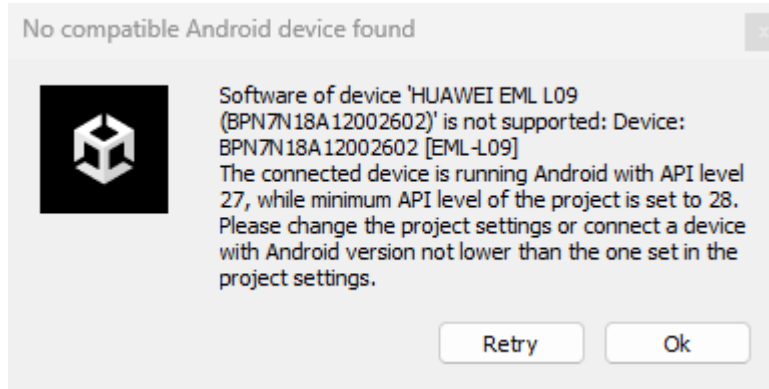
Il existe des fonctions qui agissent à titre de point d'entrée à cette boucle de rendu pour les scripts. Entre autres:

- **Awake()** est appelé une fois, avant toutes les autres fonctions et avant que la scène soit rendue. Habituellement pour initialiser des données.
- **FixedUpdate()** pour la physique (gravité, collisions).
- **Update()** pour la logique de jeu régulière.
- **LateUpdate()** pour ajuster un objet après l'exécution de toutes les autres *Updates*. Utilisé pour les caméras qui suivent des personnages par exemple.



Troubleshooting

Impossible de build à cause du *Minimum API Level*. S'assurer que le minimum est compatible avec l'appareil sur lequel on build.



Aller à Player Settings > Player > Other Settings > Minimum API Level

Liens Utiles

[AR Mobile Template Unity](#)

Pour plus de détails sur la [boucle de rendu](#).